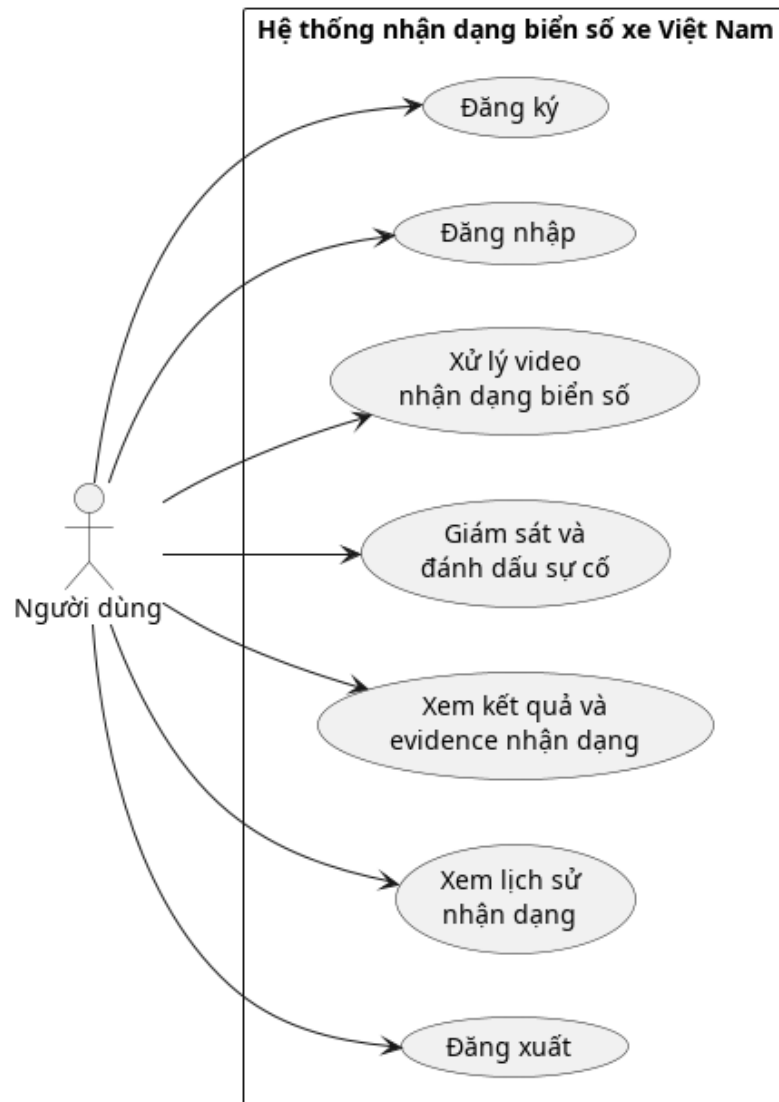


0.1 Yêu cầu và kiến trúc tổng quan

0.1.1 Sơ đồ use case



Hình 0.1: Sơ đồ usecase tổng quan

Tác nhân tham gia Hệ thống nhận dạng biển số xe Việt Nam có một tác nhân duy nhất là Người dùng (User), đại diện cho bất kỳ cá nhân nào truy cập hệ thống qua giao diện web. Tác nhân này đóng hai vai trò khác nhau tùy theo trạng thái xác thực: với các chức năng quản lý tài khoản như đăng ký và đăng nhập, người dùng là khách chưa được xác thực; với các chức năng nghiệp vụ chính như xử lý video, giám sát và xem lịch sử, người dùng phải là thành viên đã đăng nhập hợp lệ. Hệ thống không phân biệt vai trò theo nhóm quyền (admin, operator...) mà kiểm soát truy cập hoàn toàn dựa trên phiên đăng nhập.

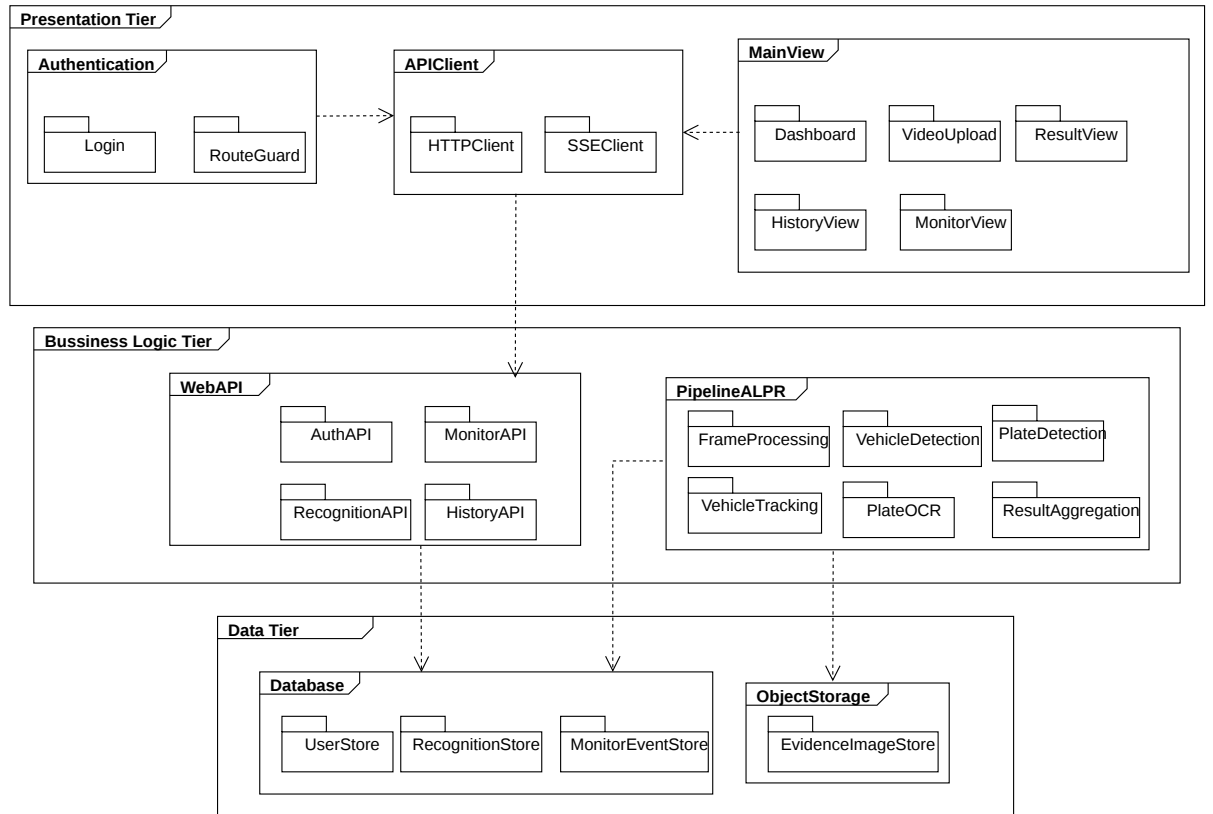
Các use case chính Hệ thống có bảy use case được mô tả trong sơ đồ, chia thành hai nhóm chức năng.

Nhóm quản lý tài khoản gồm ba use case. Đăng ký cho phép người dùng tạo tài khoản mới bằng thông tin cá nhân; hệ thống xác thực dữ liệu đầu vào, băm mật khẩu và lưu thông tin vào cơ sở dữ liệu. Đăng nhập cho phép người dùng có tài khoản xác thực danh tính và nhận JWT qua cookie bảo mật để truy cập các chức năng được bảo vệ. Đăng xuất hủy phiên làm việc hiện tại, xóa cookie và đưa người dùng về trạng thái khách.

Nhóm nghiệp vụ nhận dạng gồm bốn use case, chỉ khả dụng sau khi đăng nhập. Xử lý video nhận dạng biển số là use case trung tâm của hệ thống: người dùng tải video lên, backend khởi động pipeline ALPR đa tầng — phát hiện phương tiện, phát hiện biển số, tracking, đánh giá chất lượng crop, OCR và tổng hợp kết quả đa khung hình — rồi trả kết quả về frontend theo thời gian thực qua SSE. Giám sát và đánh dấu sự cố cho phép người dùng theo dõi luồng video liên tục (từ file upload hoặc nguồn RTSP) và đánh dấu một khoảng thời gian cụ thể để phân tích chuyên sâu, phục vụ các tình huống cần trích xuất bằng chứng. Xem kết quả và evidence nhận dạng cho phép người dùng xem lại kết quả của từng phiên xử lý, bao gồm danh sách biển số nhận dạng được, các xe bị từ chối (rejected vehicles) kèm lý do, và ảnh crop làm bằng chứng. Xem lịch sử nhận dạng cung cấp danh sách toàn bộ các phiên xử lý đã thực hiện, được lọc theo tài khoản đăng nhập, giúp người dùng tra cứu lại kết quả của các video đã xử lý trước đó.

0.1.2 Thiết kế kiến trúc

Lựa chọn kiến trúc phần mềm Hệ thống áp dụng kiến trúc ba lớp (Three-tier Architecture) kết hợp với mô hình Client–Server. Đây là kiến trúc phân tách ứng dụng thành ba tầng độc lập: tầng trình bày (Presentation Tier) chịu trách nhiệm hiển thị giao diện và tương tác với người dùng, tầng xử lý nghiệp vụ (Business Logic Tier) thực thi toàn bộ logic ứng dụng, và tầng dữ liệu (Data Tier) quản lý việc lưu trữ và truy vấn dữ liệu bền vững. Mỗi tầng chỉ giao tiếp với tầng liền kề thông qua giao diện được định nghĩa rõ ràng, giúp các tầng có thể thay đổi hoặc mở rộng độc lập mà không ảnh hưởng lẫn nhau.



Hình 0.2: Biểu đồ phụ thuộc gói

Thiết kế tổng quan

Tầng trình bày (Presentation Tier) Gồm ba package: Package Authentication chứa các màn hình xác thực người dùng, bao gồm Login (đăng nhập) và RouteGuard (bảo vệ định tuyến, ngăn người dùng chưa xác thực truy cập các trang nghiệp vụ). Package MainView tổ chức toàn bộ giao diện sau đăng nhập, gồm Dashboard (trang tổng quan), VideoUpload (tải video lên để xử lý), ResultView (xem kết quả và ảnh minh chứng của từng phiên nhận dạng), HistoryView (tra cứu lịch sử các phiên đã xử lý) và MonitorView (giám sát luồng video và đánh dấu sự kiện). Package APIClient đóng vai trò cầu nối duy nhất giữa tầng trình bày và tầng nghiệp vụ, gồm HTTPClient xử lý các yêu cầu REST đồng bộ và SSEClient nhận kết quả nhận dạng theo luồng thời gian thực từ server.

Tầng nghiệp vụ (Business Logic Tier) Gồm hai package. Package WebAPI tiếp nhận và xử lý toàn bộ yêu cầu HTTP từ client, được tổ chức thành bốn router theo chức năng: AuthAPI quản lý đăng ký, đăng nhập và xác thực JWT; RecognitionAPI

tiếp nhận video đầu vào, khởi động pipeline xử lý và phát kết quả về client qua SSE; MonitorAPI quản lý luồng giám sát và các sự kiện được đánh dấu; HistoryAPI cung cấp dữ liệu lịch sử phiên nhận dạng theo tài khoản. Package PipelineALPR thực thi toàn bộ luồng xử lý AI, gồm năm sub-package hoạt động theo chuỗi: FrameProcessing giải mã và tiền xử lý khung hình từ video đầu vào; VehicleDetection phát hiện phương tiện trong từng khung hình; VehicleTracking theo dõi từng phương tiện xuyên suốt các khung hình liên tiếp; PlateDetection phát hiện và trích xuất vùng biển số; PlateOCR nhận dạng ký tự biển số; ResultAggregation tổng hợp kết quả từ nhiều khung hình để đưa ra kết quả cuối cùng cho từng phương tiện.

Tầng dữ liệu (Data Tier) Gồm hai package. Package Database lưu trữ toàn bộ dữ liệu có cấu trúc, gồm UserStore (thông tin tài khoản và phiên xác thực), RecognitionStore (kết quả các phiên nhận dạng và bản ghi từng phương tiện) và MonitorEventStore (thông tin các sự kiện được đánh dấu trong quá trình giám sát). Package ObjectStorage lưu trữ dữ liệu nhị phân, cụ thể là EvidenceImageStore chứa ảnh crop biển số và ảnh minh chứng phương tiện; package Database chỉ lưu URL tham chiếu đến các tài nguyên này thay vì nhúng trực tiếp ảnh vào document.

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

Phần này có độ dài từ hai đến ba trang. Sinh viên đặc tả thông tin về màn hình mà ứng dụng của mình hướng tới, bao gồm độ phân giải màn hình, kích thước màn hình, số lượng màu sắc hỗ trợ, v.v. Tiếp đến, sinh viên đưa ra các thống nhất/chuẩn hóa của mình khi thiết kế giao diện như thiết kế nút, điều khiển, vị trí hiển thị thông điệp phản hồi, phối màu, v.v. Sau cùng sinh viên đưa ra một số hình ảnh minh họa thiết kế giao diện cho các chức năng quan trọng nhất. Lưu ý, sinh viên không nhầm lẫn giao diện thiết kế với giao diện của sản phẩm sau cùng.

0.2.2 Thiết kế lớp

Phần này có độ dài từ ba đến bốn trang. Sinh viên trình bày thiết kế chi tiết các thuộc tính và phương thức cho một số lớp chủ đạo/quan trọng nhất của ứng dụng (từ 2-4 lớp). Thiết kế chi tiết cho các lớp khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục.

Để minh họa thiết kế lớp, sinh viên thiết kế luồng truyền thông điệp giữa các đối tượng tham gia cho 2 đến 3 use case quan trọng nào đó bằng biểu đồ trình tự (hoặc biểu đồ giao tiếp).

0.2.3 Thiết kế cơ sở dữ liệu

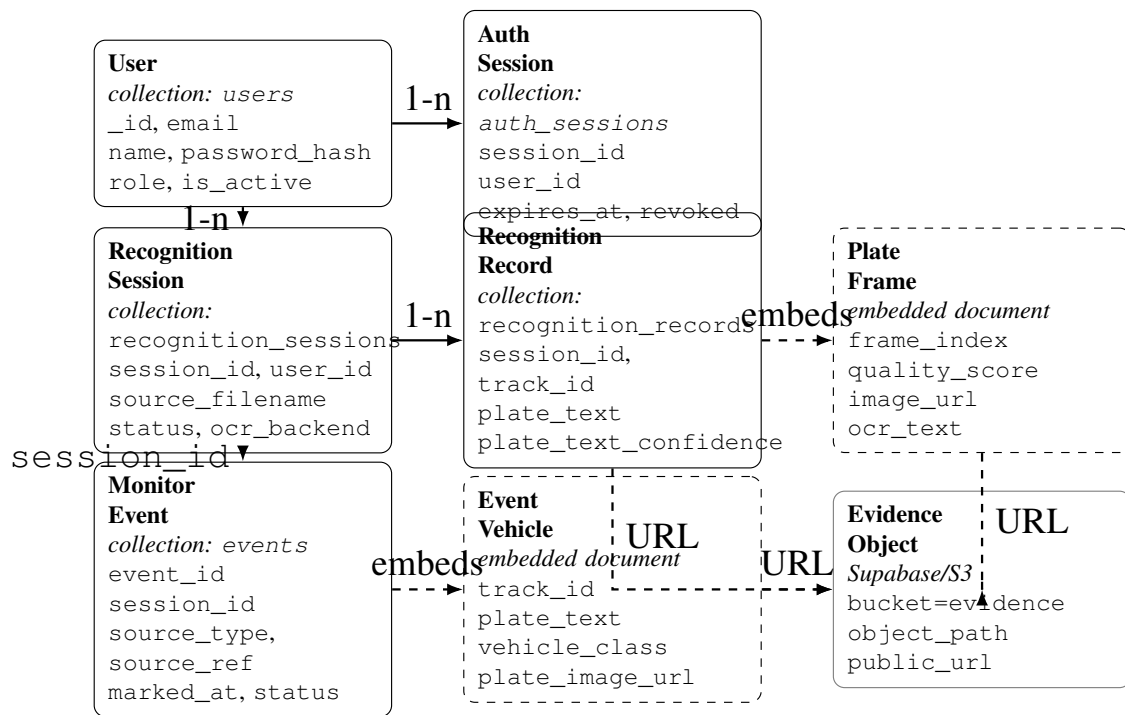
Hệ thống sử dụng MongoDB làm cơ sở dữ liệu chính và Supabase Storage/S3-compatible storage làm kho lưu trữ ảnh evidence. MongoDB chỉ lưu dữ liệu có cấu trúc: tài khoản người dùng, phiên đăng nhập, phiên xử lý video, kết quả nhận dạng và các event được đánh dấu trong chế độ monitor. Ảnh crop phương tiện, ảnh crop biển số và các frame evidence được upload lên bucket `evidence`; trong MongoDB chỉ lưu URL trỏ tới các object này.

Thiết kế này bám theo cách dữ liệu được sinh ra trong pipeline. Một phiên xử lý video có thể sinh nhiều phương tiện được theo dõi, mỗi phương tiện có một biển số cuối cùng và nhiều crop biển số theo thời gian. Nếu lưu ảnh nhị phân trực tiếp trong document, kích thước document tăng nhanh và khó mở rộng khi track kéo dài. Vì vậy, hệ thống tách metadata và ảnh evidence: MongoDB giữ khóa truy vấn, kết quả OCR và các URL; object storage giữ dữ liệu ảnh.

Về mô hình NoSQL, hệ thống kết hợp hai cách lưu. Các thực thể có vòng đời riêng và cần truy vấn độc lập được lưu thành collection riêng, ví dụ `users`, `auth_sessions`, `recognition_sessions`, `recognition_records` và `events`. Các dữ liệu nhỏ, chỉ có ý nghĩa trong ngữ cảnh document cha, được nhúng trực tiếp: `PlateFrame` nằm trong `RecognitionRecord`, còn `EventVehicle` nằm trong `MonitorEvent`. Cách này giảm số lần truy vấn khi giao diện mở chi tiết một kết quả nhận dạng hoặc một event monitor.

a, Biểu đồ thực thể liên kết logic

Mặc dù MongoDB không ràng buộc khóa ngoại như cơ sở dữ liệu quan hệ, biểu đồ E-R vẫn được dùng để mô tả quan hệ logic giữa các document chính. Các mũi tên nét liền biểu diễn quan hệ tham chiếu bằng khóa như `user_id`, `session_id`; mũi tên nét đứt biểu diễn quan hệ nhúng document con hoặc quan hệ logic không bắt buộc có khóa ngoại vật lý.



Hình 0.3: Biểu đồ E-R logic và ánh xạ NoSQL của hệ thống

Các thực thể chính trong hệ thống gồm:

- **User:** tài khoản người dùng của hệ thống. Collection triển khai là `users`.
- **AuthSession:** phiên đăng nhập phía server, dùng cùng JWT/HttpOnly cookie để xác thực request. Collection triển khai là `auth_sessions`.
- **RecognitionSession:** một phiên xử lý video hoặc nguồn dữ liệu nhận dạng. Collection triển khai là `recognition_sessions`.
- **RecognitionRecord:** kết quả cuối cùng của một phương tiện trong một phiên xử lý, gồm biển số, độ tin cậy OCR, lớp phương tiện, track id và evidence. Collection triển khai là `recognition_records`.
- **PlateFrame:** một crop biển số trong quá trình theo dõi. Đây là document con, được nhúng trong `RecognitionRecord`.
- **MonitorEvent:** event được người dùng đánh dấu trong chế độ monitor. Trong project, collection hiện tại là `events`, không còn dùng `incidents`.
- **EventVehicle:** một phương tiện được phát hiện trong cửa sổ thời gian của event. Đây là document con, được nhúng trong `MonitorEvent`.
- **EvidenceObject:** ảnh evidence lưu trong Supabase Storage/S3. Đây không phải collection MongoDB; MongoDB chỉ lưu URL.

Một User có nhiều AuthSession và nhiều RecognitionSession; cả

hai quan hệ này đều dùng trường `user_id`. Một `RecognitionSession` có nhiều `RecognitionRecord`; mỗi record dùng `session_id` để trở về phiên xử lý và dùng cặp (`session_id`, `track_id`) để tránh lưu trùng cùng một track. Đối với monitor, `MonitorEvent.session_id` dùng để gom các event thuộc cùng một phiên monitor hoặc cùng nguồn upload/live; đây là quan hệ logic trong ứng dụng, không phải khóa ngoại bắt buộc của MongoDB.

`PlateFrame` không được tách thành collection riêng vì frame chỉ có ý nghĩa khi đi cùng một record nhận dạng. Mỗi `RecognitionRecord` chứa một `best_plate_frame` và một mảng `track_buffer`. Tương tự, danh sách `EventVehicle` được nhúng trong `MonitorEvent` vì giao diện event luôn cần đọc cùng lúc trạng thái event, danh sách xe và URL ảnh evidence của từng xe.

b, Ánh xạ sang mô hình MongoDB

Từ mô hình logic trên, hệ thống triển khai thành các collection/document như Bảng 1. Các tên trong bảng được căn theo tên model và collection thật trong source code.

Bảng 1: Ánh xạ thực thể sang collection/document MongoDB

Thực thể logic	Collection/Document	Mô tả thiết kế
User	users	Collection độc lập. Lưu thông tin tài khoản, email, tên hiển thị, mật khẩu đã băm, vai trò và trạng thái hoạt động.
AuthSession	auth_sessions	Collection độc lập. Tham chiếu đến users bằng user_id; dùng để kiểm tra phiên đăng nhập và thu hồi session.
RecognitionSession	recognition_sessions	Collection độc lập. Lưu metadata của phiên xử lý như file nguồn, trạng thái, tổng số record, số frame đã xử lý, chế độ tiền xử lý và OCR backend.
RecognitionRecord	recognition_records	Collection độc lập. Tham chiếu logic đến phiên xử lý bằng session_id; lưu kết quả biển số cuối cùng của từng track.
PlateFrame	Embedded document	Được nhúng trong RecognitionRecord dưới dạng best_plate_frame và mảng track_buffer.
MonitorEvent	events	Collection độc lập. Lưu event được đánh dấu trong chế độ monitor, gồm nguồn, khoảng thời gian, trạng thái xử lý và danh sách phương tiện trong event.
EventVehicle	Embedded document	Được nhúng trong MonitorEvent dưới dạng mảng vehicles.
EvidenceObject	Supabase/S3 object	Không phải collection MongoDB. File ảnh được lưu trong bucket object storage; MongoDB chỉ lưu URL trong các trường image_url, vehicle_thumbnail_url, plate_image_url, vehicle_image_url.

c, Thiết kế các collection chính

Collection users. Collection này lưu tài khoản người dùng. Các trường chính gồm _id, email, name, password_hash, role, is_active, created_at và updated_at. Trường email được đặt unique index để không có hai tài khoản dùng cùng một email. Hệ thống không lưu mật khẩu gốc.

Collection auth_sessions. Collection này lưu phiên đăng nhập phía server. Mỗi document gồm session_id, user_id, expires_at, revoked, created_at và updated_at. Khi đăng nhập, backend tạo một session, sinh JWT

chứa `user_id` và `session_id`, sau đó gửi về trình duyệt bằng `HttpOnly` cookie. Khi xác thực request, backend giải mã JWT và đối chiếu với document trong `auth_sessions`.

Collection `recognition_sessions`. Collection này biểu diễn một phiên xử lý video hoặc nguồn nhận dạng. Các trường chính gồm `session_id`, `user_id`, `source_filename`, `source_type`, `status`, `total_records`, `processed_frames`, `preprocess_mode`, `ocr_backend`, `error_message`, `created_at` và `updated_at`. Khi người dùng upload video, backend tạo session ở trạng thái `processing`; khi pipeline kết thúc, session được cập nhật thành `completed` hoặc `failed`.

Collection `recognition_records`. Collection này lưu kết quả cuối cùng cho từng phương tiện được theo dõi. Các trường định danh gồm `session_id`, `user_id`, `track_id`, `vehicle_track_id`, `plate_track_id` và `vehicle_class`. Các trường OCR gồm `plate_text`, `plate_text_confidence`, `ocr_vote_summary` và `ocr_method`. Các trường thời gian gồm `first_seen_frame`, `last_seen_frame`, `created_at` và `updated_at`.

Trong `recognition_records`, phần `evidence` được tổ chức thành ba nhóm. Trường `vehicle_thumbnail_url` lưu URL ảnh phương tiện. Trường `best_plate_frame` lưu crop biển số tốt nhất dưới dạng `PlateFrame`. Trường `track_buffer` là mảng `PlateFrame`, mỗi phần tử gồm `frame_index`, `quality_score`, `image_url`, `timestamp_ms`, `plate_bbox`, `detection_confidence`, `ocr_text` và `ocr_confidence`. Đây là dữ liệu phục vụ màn hình xem `evidence` và kiểm tra lại các frame đã đóng góp vào kết quả OCR.

Collection `events`. Collection này thay thế mô hình `incidents` cũ. Mỗi document `MonitorEvent` lưu một cửa sổ thời gian được người dùng đánh dấu trong chế độ monitor. Các trường chính gồm `event_id`, `session_id`, `source_type`, `source_ref`, `marked_at`, `window_start_sec`, `window_end_sec`, `duration_sec`, `status`, `vehicles`, `total_vehicles`, `processing_ms`, `error_message`, `created_at` và `updated_at`. Trường `vehicles` là mảng `EventVehicle`. Mỗi phần tử lưu `track_id`, `vehicle_track_id`, `plate_track_id`, `plate_text`, `plate_text_confidence`, `chars`, `vehicle_class`, `plate_image_url`, `vehicle_image_url`, `ocr_method`, `ocr_frames`, `first_seen_frame` và `last_seen_frame`.

d, Thiết kế lưu trữ `evidence` bằng Supabase Storage/S3

Các ảnh `evidence` được lưu vào bucket `evidence`. Trong luồng xử lý video thông thường, hệ thống lưu ảnh theo mẫu đường dẫn:

- `{session_id}/track_{track_id}_frame_{index}.jpg`: các frame trong track buffer;
- `{session_id}/plate_{track_id}.jpg`: ảnh biển số tốt nhất;
- `{session_id}/vehicle_{track_id}.jpg`: ảnh crop phương tiện.

Trong luồng monitor event, ảnh evidence được lưu theo mẫu:

- `events/{event_id}/plate_{id}.jpg`;
- `events/{event_id}/vehicle_{id}.jpg`.

Sau khi upload ảnh, backend lấy public URL và lưu URL đó vào MongoDB. Nếu upload không thành công, URL có thể rỗng nhưng kết quả nhận dạng vẫn được lưu, giúp pipeline không phụ thuộc tuyệt đối vào object storage. Nếu triển khai trong môi trường cần bảo mật evidence chặt hơn, bucket có thể chuyển sang private và backend sinh signed URL có thời hạn thay vì lưu public URL.

e, Thiết kế chỉ mục

Các chỉ mục được tạo trong hàm khởi tạo MongoDB để phục vụ đúng các truy vấn chính của ứng dụng: đăng nhập bằng email, kiểm tra session, liệt kê lịch sử theo người dùng, lấy record theo phiên và lấy event gần nhất. Bảng 2 liệt kê các chỉ mục chính.

Bảng 2: Các chỉ mục MongoDB chính

Collection	Chỉ mục	Mục đích
users	email unique; created_at desc	Đảm bảo email không trùng, tăng tốc đăng nhập và hỗ trợ liệt kê tài khoản mới.
auth_sessions	session_id unique; user_id; expires_at	Kiểm tra phiên đăng nhập, truy vết phiên theo người dùng và hỗ trợ xử lý phiên hết hạn.
recognition_sessions	session_id unique; (user_id, created_at); status; created_at desc	Lấy phiên theo mã, liệt kê lịch sử gần nhất của người dùng và lọc theo trạng thái xử lý.
recognition_records	session_id; (user_id, session_id); plate_text; vehicle_track_id; plate_track_id; vehicle_class; created_at desc; (session_id, track_id) unique	Lấy record của một phiên, lấy record theo track, tìm kiếm biển số, lọc theo mã track phương tiện/biển số và tránh lưu trùng cùng một track trong một session.
events	event_id unique; session_id; marked_at desc; status; source_type	Lấy chi tiết event, liệt kê event theo phiên monitor, theo nguồn và theo thời gian đánh dấu.

f, Luồng truy vấn chính

Thiết kế collection và chỉ mục được chọn dựa trên các luồng truy vấn chính của ứng dụng. Khi người dùng đăng nhập, hệ thống tìm tài khoản theo email, kiểm tra mật khẩu và tạo document trong auth_sessions. Khi người dùng mở lịch sử, frontend gọi API lấy danh sách recognition_sessions theo user_id, sắp xếp giảm dần theo created_at. Khi người dùng chọn một phiên, backend lấy các recognition_records theo session_id và user_id. Khi người dùng mở track buffer, backend lấy một record theo cặp (session_id, track_id).

Đối với monitor, khi người dùng đánh dấu một khoảng thời gian, backend tạo event_id, chạy pipeline ALPR trên cửa sổ đó, phát tiến độ qua SSE tại /moni-

tor/{session_id}/events/stream, rồi lưu kết quả vào collection events. Khi cần xem lại, backend cung cấp API /events/{event_id} để lấy một event và /events để liệt kê event theo session_id, source_type hoặc giới hạn số lượng. Vì danh sách EventVehicle được nhúng trực tiếp trong document MonitorEvent, giao diện có thể hiển thị chi tiết event bằng một lần đọc document chính.

Tóm lại, thiết kế cơ sở dữ liệu hiện tại không còn sử dụng collection incidents. Toàn bộ luồng monitor được chuẩn hóa theo khái niệm events. MongoDB lưu metadata và kết quả nhận dạng, còn ảnh evidence được tách sang object storage để giảm kích thước document và giữ cho các truy vấn lịch sử, chi tiết record và chi tiết event đơn giản.

0.3 Xây dựng ứng dụng

Thư viện và công cụ sử dụng

Hệ thống được phát triển bằng các công cụ, ngôn ngữ lập trình và thư viện chính trong Bảng 3.

Bảng 3: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ/Thư viện	Phiên bản	Địa chỉ URL
IDE lập trình	Visual Studio Code	1.104.0	https://code.visualstudio.com/
Quản lý mã nguồn	Git	2.43.0	https://git-scm.com/
Đóng gói ứng dụng	Docker	28.5.1	https://www.docker.com/
Ngôn ngữ backend	Python	3.10	https://www.python.org/
Runtime frontend	Node.js	20	https://nodejs.org/
Package manager	npm	11.6.3	https://www.npmjs.com/
Backend API	FastAPI	$\geq 0.111.0$	https://fastapi.tiangolo.com/
ASGI server	Uvicorn	$\geq 0.29.0$	https://www.uvicorn.org/

Mục đích	Công cụ/Thư viện	Phiên bản	Địa chỉ URL
Học sâu	PyTorch	2.4.1	https://pytorch.org/
Xử lý ảnh	OpenCV Python	4.10.0.84	https://opencv.org/
Phát hiện đối tượng	Ultralytics YOLO	$\geq 8.0.0$	https://docs.ultralytics.com/
Theo dõi đối tượng	BoxMOT	19.0.0	https://github.com/mikel-brostrom/boxmot
Suy luận ONNX	ONNX Runtime	1.20.0	https://onnxruntime.ai/
Tính toán ma trận	NumPy	1.26.4	https://numpy.org/
Tính toán khoa học	SciPy	$\geq 1.11.0$	https://scipy.org/
Huấn luyện mô hình	Lightning	2.x	https://lightning.ai/
Xử lý ảnh phụ trợ	Pillow	$\geq 10.0.0$	https://python-pillow.org/
Đọc cấu hình	PyYAML	$\geq 6.0.1$	https://pyyaml.org/
Xác thực mật khẩu	bcrypt	$\geq 4.1.0$	https://github.com/pyca/bcrypt
JWT	PyJWT	$\geq 2.8.0$	https://pyjwt.readthedocs.io/
Cơ sở dữ liệu	MongoDB	7	https://www.mongodb.com/
Driver MongoDB	Motor	$\geq 3.3.0$	https://motor.readthedocs.io/
Lưu trữ ảnh	Supabase	$\geq 2.5.0$	https://supabase.com/
Frontend framework	React	18.3.1	https://react.dev/

Mục đích	Công cụ/Thư viện	Phiên bản	Địa chỉ URL
Điều hướng frontend	React Router DOM	7.17.0	https://reactrouter.com/
Build frontend	Vite	5.4.21	https://vite.dev/
Thiết kế giao diện	Tailwind CSS	3.4.19	https://tailwindcss.com/
Streaming video	MediaMTX	v1.19.1	https://github.com/bluenviron/mediamtx
Xử lý video	FFmpeg	6.1.1	https://ffmpeg.org/
Kiểm thử	pytest	–	https://docs.pytest.org/

0.3.1 Frontend

Frontend của hệ thống được xây dựng dưới dạng ứng dụng web một trang bằng React và Vite, mã nguồn đặt trong thư mục `web/src`. Giao diện không thực hiện suy luận mô hình trực tiếp, mà đóng vai trò tiếp nhận đầu vào từ người dùng, gửi yêu cầu tới backend, nhận tiền độ xử lý qua cơ chế streaming và hiển thị kết quả nhận dạng theo từng phương tiện hoặc từng event.

Ứng dụng được tổ chức quanh hai chế độ chính: chế độ xử lý video tải lên và chế độ giám sát. Tại `App.jsx`, trạng thái mode nhận hai giá trị `process` và `monitor`; người dùng chuyển chế độ bằng thành phần `SegmentedControl` trên thanh điều hướng. Cách tổ chức này giúp tách rõ hai luồng sử dụng: luồng `process` dùng cho đánh giá một video hoàn chỉnh, còn luồng `monitor` dùng cho việc kết nối camera hoặc mở video và đánh dấu một khoảng thời gian cần phân tích.

Thành phần	Vai trò trong giao diện
AuthProvider, apiClient.js	Quản lý trạng thái đăng nhập, gọi các API /auth/me, /auth/login, /auth/register, /auth/logout và tự động gắn CSRF token cho các yêu cầu ghi dữ liệu đi qua apiFetch.
DashboardPage	Điều phối trạng thái chính của màn hình làm việc: file video, job_id, tiến độ xử lý, danh sách phương tiện hợp lệ, danh sách kết quả bị loại và lỗi trả về từ backend.
SourcePanel	Nhận file video, chọn chế độ tiền xử lý ảnh và backend OCR trước khi gửi yêu cầu POST /upload.
MediaStage	Hiển thị video gốc hoặc khung hình preview được backend gửi về trong quá trình xử lý.
ResultsPanel, VehicleCard	Hiển thị kết quả theo từng track phương tiện, gồm biến số tổng hợp, độ tin cậy, ảnh evidence và các khung hình trong track buffer.
HistoryModal	Truy vấn lại danh sách phiên và các record đã lưu thông qua /sessions và /sessions/{session_id}/records.
MonitorPage	Quản lý phiên giám sát RTSP hoặc video upload, gửi lệnh đánh dấu event và nhận tiến độ phân tích event qua SSE.
LiveViewer, UploadViewer, EventsPanel	Hiển thị luồng camera/video, chọn khoảng thời gian cần phân tích và trình bày danh sách event đã đánh dấu.

Bảng 4: Các thành phần frontend chính

Trong luồng xử lý video tải lên, người dùng chọn file tại SourcePanel. Hook useUpload đóng gói file vào FormData, kèm theo preprocess_mode và ocr_backend, sau đó gọi POST /upload. Backend trả về job_id; từ thời điểm này, hook useStream mở kết nối EventSource tới /stream/{job_id} để nhận các sự kiện progress, frame, vehicle, rejected_vehicle, complete và error. Dữ liệu nhận được được cập nhật vào state của DashboardPage, nhờ vậy giao diện có thể hiển thị tiến độ, khung hình đang xử lý và kết quả nhận dạng mà không cần tải lại trang.

Trong luồng giám sát, MonitorPage cho phép người dùng chọn một trong hai nguồn: RTSP camera hoặc video upload. Với RTSP, frontend gửi rtsp_url tới POST /monitor/live/connect và nhận lại session_id, whep_url, mjpeg_url. Với video upload, frontend gửi file tới POST /monitor/upload và nhận URL phát lại video. Khi người dùng đánh dấu một thời điểm hoặc một

khoảng thời gian, frontend gọi `POST /monitor/{session_id}/mark`; kết quả phân tích được đẩy ngược về qua `/monitor/{session_id}/events/stream`. Cách này giúp thao tác đánh dấu event nhanh, trong khi quá trình ALPR vẫn chạy bất đồng bộ ở backend.

Về mặt trải nghiệm, frontend ưu tiên hiển thị theo tiến trình thực tế của pipeline. Các trạng thái rỗng, đang upload, đang xử lý, hoàn thành và lỗi được tách riêng bằng các thành phần giao diện như `Badge`, `Toast`, `Progress` và `EmptyState`. Điều này giúp người dùng theo dõi được hệ thống đang ở bước nào: chờ chọn nguồn, đang tải video, đang phân tích, đã có kết quả hoặc cần xử lý lỗi đầu vào.

0.3.2 Backend và API

Backend được xây dựng bằng FastAPI, mã nguồn chính đặt trong thư mục `api`. File `api/main.py` chỉ giữ vai trò định nghĩa route, kiểm tra đầu vào, quản lý hàng đợi streaming và điều phối job xử lý. Các phần tính toán nặng được tách sang thư mục `api/core`, gồm nạp mô hình, phát hiện phương tiện, phát hiện biển số, theo dõi đối tượng, OCR, chấm điểm chất lượng khung hình và tổng hợp kết quả theo track.

Khi ứng dụng khởi động, hàm `lifespan` của FastAPI nạp các mô hình một lần thông qua `load_models()`, khởi tạo kết nối MongoDB nếu có cấu hình `MONGODB_URI`, đồng thời bật tác vụ dọn dẹp các phiên monitor tạm thời. Cách nạp mô hình ở giai đoạn khởi động giúp tránh việc mỗi request upload lại phải khởi tạo YOLO, OCR hoặc ReID từ đầu.

Nhóm API	Endpoint và chức năng
Xác thực	/auth/register, /auth/login, /auth/logout, /auth/me, /auth/csrf. Nhóm API này tạo tài khoản, đăng nhập, đăng xuất, kiểm tra phiên người dùng và cấp CSRF token cho các yêu cầu ghi dữ liệu.
Xử lý video tải lên	POST /upload tạo job xử lý video; GET /stream/{job_id} trả tiến độ và kết quả qua SSE; GET /stream/{job_id}/mjpeg trả khung hình preview dạng MJPEG; GET /records/{job_id}/{track_id} lấy lại kết quả chi tiết của một track.
Lịch sử nhận dạng	GET /sessions, GET /sessions/{session_id}, GET /sessions/{session_id}/records. Nhóm API này phục vụ màn hình lịch sử và chỉ trả dữ liệu thuộc về người dùng hiện tại.
Giám sát và event	POST /monitor/upload, POST /monitor/live/connect, POST /monitor/{session_id}/mark, GET /monitor/{session_id}/events/stream, GET /events/{event_id}, GET /events. Nhóm API này quản lý phiên monitor, đánh dấu event và truy vấn lại kết quả event đã lưu.

Bảng 5: Các nhóm API chính của backend

Luồng POST /upload được thiết kế theo cơ chế bất đồng bộ. Backend kiểm tra định dạng file theo phần mở rộng .mp4, .avi, .webm, .mov, .mkv, kiểm tra kiểu nội dung, dung lượng tối đa và file rỗng. Sau đó hệ thống sinh job_id, tạo hàng đợi SSE, ghi video vào file tạm và đưa hàm run_job vào executor. Mỗi job được gắn với user_id; các endpoint /stream/{job_id} và /stream/{job_id}/mjpeg đều kiểm tra chủ sở hữu job trước khi trả dữ liệu.

Trong quá trình chạy pipeline, backend đẩy các bản tin JSON vào hàng đợi của job. Các bản tin này gồm tiến độ theo frame, khung hình preview, kết quả phương tiện hợp lệ, kết quả bị loại do không đạt điều kiện hậu xử lý, trạng thái hoàn thành và lỗi. FastAPI chuyển các bản tin này thành luồng text/event-stream, nhờ đó frontend có thể cập nhật giao diện gần thời gian thực mà không cần polling liên tục.

Nhóm API xác thực được triển khai trong api/auth.py. Mật khẩu người

dùng được băm bằng bcrypt, phiên đăng nhập được lưu trong collection `auth_sessions`, còn trình duyệt nhận JWT trong cookie `HttpOnly`. Với các API ghi dữ liệu thuộc luồng xử lý video chính, backend yêu cầu CSRF token trong header `X-CSRF-Token`. Cách triển khai này tách dữ liệu xác thực khỏi mã frontend và tránh lưu token truy cập trực tiếp trong `localStorage`.

Đối với chức năng giám sát, file `api/routes_monitor.py` quản lý hai loại phiên: `upload` và `live`. Phiên `upload` lưu video tạm để người dùng chọn khoảng thời gian cần phân tích; phiên `live` tạo `LiveSession`, kết nối nguồn RTSP và phát lại qua MediaMTX. Khi người dùng đánh dấu event, backend tạo `event_id`, dựng `FrameSource` tương ứng với nguồn video, rồi gọi `run_event` trong `api/core/event_analyzer.py`. Kết quả event được phát về frontend qua SSE và được lưu vào collection `events`.

Lớp lưu trữ được tách khỏi route xử lý chính. MongoDB lưu thông tin người dùng, phiên xử lý, record nhận dạng và event monitor thông qua các hàm trong `api/database/mongodb.py`. Supabase Storage được sử dụng cho ảnh evidence của track và event để tránh lưu ảnh trực tiếp trong document MongoDB. Nhờ cách tách này, backend API giữ vai trò điều phối, còn dữ liệu lớn và metadata được lưu ở các thành phần phù hợp hơn.

0.3.3 Kết quả đạt được

Sinh viên trước tiên mô tả kết quả đạt được của mình là gì, ví dụ như các sản phẩm được đóng gói là gì, bao gồm những thành phần nào, ý nghĩa, vai trò?

Sinh viên cần thống kê các thông tin về ứng dụng của mình như: số dòng code, số lớp, số gói, dung lượng toàn bộ mã nguồn, dung lượng của từng sản phẩm đóng gói, v.v. Tương tự như phần liệt kê về công cụ sử dụng, sinh viên cũng nên dùng bảng để mô tả phần thông tin thống kê này.

0.3.4 Minh họa các chức năng chính

Sinh viên lựa chọn và đưa ra màn hình cho các chức năng chính, quan trọng, và thú vị nhất. Mỗi giao diện cần phải có lời giải thích ngắn gọn. Khi giải thích, sinh viên có thể kết hợp với các chú thích ở trong hình ảnh giao diện.

0.4 Kiểm thử

Phần này có độ dài từ hai đến ba trang. Sinh viên thiết kế các trường hợp kiểm thử cho hai đến ba chức năng quan trọng nhất. Sinh viên cần chỉ rõ các kỹ thuật kiểm thử đã sử dụng. Chi tiết các trường hợp kiểm thử khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục. Sinh viên sau cùng tổng kết về số lượng các trường hợp kiểm thử và kết quả kiểm thử. Sinh viên cần phân tích lý do nếu kết quả kiểm

thử không đạt.

0.5 Triển khai

Ứng dụng được triển khai theo mô hình tách riêng frontend, backend xử lý AI và các dịch vụ lưu trữ. Frontend được triển khai trên Vercel, backend được đóng gói bằng Docker và chạy trên RunPod GPU. Cơ sở dữ liệu và lưu trữ ảnh kết quả sử dụng các dịch vụ bên ngoài để giảm tải cho server xử lý chính.

Thành phần	Cấu hình triển khai
Frontend	React/Vite, triển khai trên Vercel
Backend API	FastAPI, chạy trên cổng 8000
Server xử lý AI	RunPod GPU, NVIDIA RTX 3090
Môi trường GPU	PyTorch 2.4.1, CUDA 12.1, cuDNN 9
Streaming video	MediaMTX, sử dụng WebRTC/RTSP
Cơ sở dữ liệu	MongoDB Atlas
Lưu trữ ảnh kết quả	Supabase Storage
Giới hạn dung lượng upload	512 MB

Bảng 6: Cấu hình triển khai ứng dụng

Trong môi trường thử nghiệm cục bộ, hệ thống được chạy bằng Docker Compose gồm bốn dịch vụ chính: backend API, frontend web, MediaMTX và MongoDB. Backend mở cổng 8000, frontend mở cổng 3000, MongoDB mở cổng 27017, MediaMTX sử dụng các cổng 8554, 8889, 8189/udp và 9997. Cấu hình này phục vụ kiểm thử chức năng đăng nhập, tải video, xử lý nhận dạng và xem kết quả trực tiếp.

Trong môi trường triển khai thử nghiệm trên server GPU, backend và MediaMTX được đóng gói trong cùng một Docker image. Backend đảm nhiệm các bước phát hiện phương tiện, phát hiện biển số, tracking, OCR và trả kết quả qua API. MediaMTX phục vụ luồng video trực tiếp, hỗ trợ hiển thị kết quả giám sát trên giao diện web. Các mô hình như detector phương tiện, detector biển số, OCR và ReID được đặt trên network volume của RunPod.